

Manual del script de restauración de configuraciones

Topología de monitorización · Python + Netmiko + Paramiko + GitHub

Este manual explica el script utilizado para descargar configuraciones desde GitHub y restaurarlas en routers de una topología GNS3. Está pensado como guía rápida para un examen, indicando qué partes son generales y cuáles dependen de la topología.

1. ¿Qué hace el script?

El programa realiza este proceso: 1) pregunta qué tipo de router se quiere restaurar; 2) pide IP, usuario y contraseña; 3) descarga el fichero correspondiente desde GitHub; 4) se conecta al router mediante SSH usando Netmiko; 5) aplica la configuración con el método específico de cada sistema; y 6) cierra la conexión.

Idea para memorizar: seleccionar → pedir datos → descargar → conectar → restaurar → guardar → desconectar.

2. Librerías

```
import io
import urllib.request
from getpass import getpass
```

```
import paramiko
from netmiko import ConnectHandler
```

io: permite trabajar con el contenido del fichero en memoria. Se utiliza especialmente para subir el fichero a MikroTik mediante SFTP.

urllib.request: descarga la configuración desde la URL de GitHub.

getpass: solicita la contraseña sin mostrarla en pantalla.

paramiko: biblioteca SSH. En este script se usa para compatibilidad de algoritmos y para acceder al transporte SSH/SFTP.

netmiko: simplifica la conexión y el envío de comandos a diferentes tipos de dispositivos de red.

3. Compatibilidad SSH

```
for k in (
    "diffie-hellman-group14-sha1",
    "diffie-hellman-group-exchange-sha1"
):
    if k in paramiko.Transport._kex_info:
        if k not in paramiko.Transport._preferred_kex:
            paramiko.Transport._preferred_kex += (k,)
```

Este bloque añade algoritmos Diffie-Hellman antiguos si están disponibles en Paramiko. Su finalidad es evitar problemas de negociación SSH con determinados routers o imágenes de GNS3.

En el examen: normalmente no tienes que modificarlo.

4. Ruta de GitHub

```
BASE = (
    "https://raw.githubusercontent.com/"
    "vbarraganescudero-tech/configuraciones-router/"
    "main/Router_monitorización/"
)
```

BASE es la URL base desde la que se descargan los ficheros. El programa después añade el nombre del fichero seleccionado.

Esta es una de las partes que más probablemente tendrás que cambiar en un examen.

Ejemplo actual:

BASE + archivo
→ https://raw.githubusercontent.com/.../Router_monitorización/router.cfg

Si el examen utiliza otro repositorio, otra rama o una carpeta diferente, cambia BASE. No cambies la URL para que apunte a la página normal de GitHub: debe apuntar al contenido raw del fichero.

5. Diccionario ROUTERS

```
ROUTERS = {  
    "cisco": ("cisco_ios", "router.cfg"),  
    "mikrotik": ("mikrotik_routeros", "mikrotik.rsc"),  
    "vyos": ("vyos", "config.boot"),  
    "frr": ("linux", "frr.conf"),  
}
```

Cada entrada tiene dos datos: el nombre que escribe el usuario y el tipo de dispositivo que entiende Netmiko, seguido del nombre del fichero que debe descargar.

Ejemplo: si el usuario escribe cisco, Netmiko utiliza cisco_ios y el programa descarga router.cfg.

Si el examen usa menos routers

Puedes eliminar las entradas que no aparezcan en la topología. Por ejemplo, si solo hay Cisco y VyOS:

```
ROUTERS = {  
    "cisco": ("cisco_ios", "router.cfg"),  
    "vyos": ("vyos", "config.boot"),  
}
```

Si aparece un router nuevo, se añade una entrada con su tipo Netmiko y su fichero.

6. Datos solicitados al usuario

```
router = input(  
    "Router (cisco/mikrotik/vyos/frr): "  
)  
.lower()  
  
ip = input("IP: ")  
usuario = input("Usuario: ")  
password = getpass("Contraseña: ")
```

El programa no tiene las IP ni las contraseñas escritas en el código. Las solicita al ejecutar el programa. `.lower()` convierte la elección a minúsculas.

Por ejemplo, si la topología del examen tiene IP diferentes, no hay que modificar el código: se escribe la IP correcta al ejecutar el programa.

7. Comprobación del router

```
if router not in ROUTERS:  
    print("[ERROR] Router no válido.")  
    exit()  
  
tipo, archivo = ROUTERS[router]
```

Comprueba que el nombre introducido existe en ROUTERS. Después obtiene el tipo Netmiko y el nombre del fichero correspondiente.

8. Descarga desde GitHub

```
try:  
    texto = urllib.request.urlopen(  
        BASE + archivo  
    ).read().decode()  
  
    print("[OK] Configuración descargada.")  
  
except Exception as e:  
    print("[ERROR] GitHub:", e)
```

```
exit()
```

Construye la URL uniendo BASE con archivo, descarga el contenido, lo convierte a texto y lo guarda en la variable texto. Si hay un problema de conexión o de URL, muestra un error y termina.

9. Conexión mediante Netmiko

```
conexion = ConnectHandler(  
    device_type=tipo,  
    host=ip,  
    username=usuario,  
    password=password  
)
```

ConnectHandler crea la conexión SSH utilizando el tipo de dispositivo, IP, usuario y contraseña. La variable conexion se utiliza después para enviar comandos.

10. Restauración en Cisco

Cisco es diferente porque el fichero contiene elementos que no conviene enviar directamente, como la versión, separadores, banners y bloques line. El script filtra esas partes.

```
lineas = []

ignorar_banner = False
ignorar_line = False

for linea in texto.splitlines():

    linea = linea.strip()

    if not linea:
        continue
```

splitlines() separa el fichero por líneas. strip() elimina espacios al principio y al final. Las variables ignorar_banner e ignorar_line sirven para saber si el programa está dentro de un bloque que debe saltarse.

Banners:

```
if linea.startswith("banner "):
    ignorar_banner = True
    continue

if ignorar_banner:
    if linea.startswith("!"):
        ignorar_banner = False
    continue
```

Ignora el contenido del banner hasta encontrar la línea separadora.

Líneas de consola, AUX y VTY:

```
if (
    linea.startswith("line con") or
    linea.startswith("line aux") or
    linea.startswith("line vty")
):
    ignorar_line = True
    continue

if ignorar_line:
    if linea.startswith("!"):
        ignorar_line = False
    continue
```

Evita enviar esos bloques al router durante la restauración.

Filtrado adicional:

```
if linea == "end":
    continue

if linea.startswith("version "):
    continue

if linea.startswith("!"):
    continue

lineas.append(linea)
```

El resultado se envía con send_config_set(). Después se intenta guardar la configuración.

```
salida = conexion.send_config_set(
    lineas,
    cmd_verify=False
)

try:
    conexion.save_config()
except:
    pass
```

11. Restauración en MikroTik

En MikroTik el script no envía línea por línea. Sube primero el fichero .rsc al router mediante SFTP y después ejecuta el import.

```
sftp = conexion.remote_conn_pre.open_sftp()

sftp.putfo(
    io.BytesIO(texto.encode()),
    "restaurar.rsc"
)

sftp.close()
```

El contenido descargado se convierte en un flujo de datos y se copia al router como restaurar.rsc.

```
salida = conexion.send_command_timing(
    "/import file-name=restaurar.rsc",
    read_timeout=180
)

conexion.send_command_timing(
    'file remove [find name="restaurar.rsc"]'
)
```

Después importa el fichero y finalmente elimina el fichero temporal del router.

Importante: este bloque ya está probado en tu práctica. No lo cambiaría en el examen salvo que cambie el formato o el método exigido.

12. Restauración en VyOS

VyOS utiliza un fichero de configuración. El script crea /tmp/config.boot, escribe allí el contenido descargado y después ejecuta load, commit y save.

```
conexion.send_command(
    ": > /tmp/config.boot",
    cmd_verify=False
)
```

Vacía o crea el fichero temporal.

```
for linea in texto.splitlines():

    if linea.strip():

        linea = linea.replace(
            '"',
            '\"'
        )

        conexion.send_command(
            "echo '" + linea + "' >> /tmp/config.boot",
            cmd_verify=False
        )
```

Escribe cada línea en el fichero. El replace protege las comillas simples para que el comando echo no se rompa.

```
salida = conexion.send_config_set(
    [
        "load /tmp/config.boot",
        "commit",
        "save"
    ]
)
```

load carga la configuración, commit la aplica y save la guarda para que permanezca después de reiniciar.

13. Restauración en FRR

FRR sigue una idea parecida a VyOS: crea un fichero temporal, copia la configuración y después la carga mediante vtysh.

```
conexion.send_command(
    ": > /tmp/frr.conf",
    cmd_verify=False
)
```

```

for linea in texto.splitlines():

    if linea.strip():

        linea = linea.replace(
            '"',
            '\"'
        )

        conexion.send_command(
            "echo '" + linea + "' >> /tmp/frr.conf",
            cmd_verify=False
        )

salida = conexion.send_command(
    "vtysh -f /tmp/frr.conf"
)

conexion.send_command(
    "vtysh -c 'write memory'"
)

```

vtysh -f carga el fichero de configuración y write memory guarda la configuración de FRR.

14. Cierre de la conexión

```

conexion.disconnect()

print()
print("[OK] Proceso terminado.")

except Exception as e:
    print()
    print("[ERROR]", e)

```

disconnect() cierra la conexión SSH. El bloque except captura errores producidos durante la conexión o restauración y muestra el motivo.

15. ¿Qué cambiar si el examen tiene otra topología?

Elemento	¿Cambiar?	Qué hacer
IP	Normalmente no	Se introduce al ejecutar el programa.
Usuario	Normalmente no	Se introduce al ejecutar el programa.
Contraseña	Normalmente no	Se introduce al ejecutar el programa.
Repositorio GitHub	Sí, si cambia	Modificar BASE.
Carpeta de GitHub	Sí, si cambia	Modificar BASE.
Rama	Sí, si cambia	Modificar BASE, por ejemplo main.
Nombre de fichero	Sí, si cambia	Modificar ROUTERS.
Cantidad de routers	Sí	Eliminar de ROUTERS los que no existan.
Tipo Netmiko	Solo si cambia dispositivo	Cambiar el primer valor de ROUTERS.
Bloques de restauración	Solo si cambia sistema	Mantener los bloques de los routers utilizados.

16. Ejemplo: examen con menos routers

Supongamos que en el examen solo existen Cisco y FRR y los archivos se llaman cisco.cfg y frr.cfg.

```
BASE = (  
    "https://raw.githubusercontent.com/"  
    "usuario/repositorio/"  
    "main/Router_monitorizacion/"  
)  
  
ROUTERS = {  
    "cisco": ("cisco_ios", "cisco.cfg"),  
    "frr": ("linux", "frr.cfg"),  
}
```

No sería necesario borrar los bloques de MikroTik o VyOS para que el programa funcione, pero para un examen puedes eliminarlos si no forman parte de la topología. Lo importante es que ROUTERS y los bloques if/elif se correspondan con los dispositivos que vas a utilizar.

17. Ejemplo: cambiar solo los nombres de archivos

Si el profesor mantiene los mismos cuatro tipos de router pero entrega otros nombres:

```
ROUTERS = {  
    "cisco": ("cisco_ios", "Albania.cfg"),  
    "mikrotik": ("mikrotik_routers", "Angola.rsc"),  
    "vyos": ("vyos", "Argentina.boot"),  
    "frr": ("linux", "Australia.conf"),  
}
```

Los bloques de restauración no se modifican, porque siguen siendo los mismos sistemas operativos.

18. Ejemplo: añadir JunOS

Si el examen añade JunOS y el fichero es junos.conf, habría que añadirlo a ROUTERS y disponer de su bloque de restauración específico. JunOS no debe tratarse como Cisco línea por línea porque su configuración puede estar en formato jerárquico con llaves.

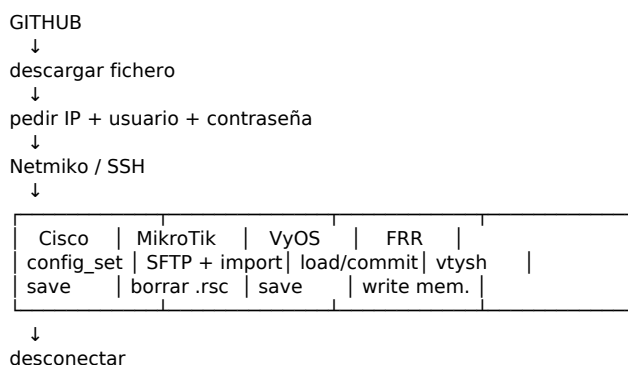
```
"junos": ("juniper_junos", "junos.conf")
```

En tu práctica anterior se utilizó SFTP para subir el fichero y después load override y commit para JunOS.

19. Lo que debes memorizar para el examen

- BASE = dónde están los ficheros de GitHub.
- ROUTERS = qué routers existen, qué tipo reconoce Netmiko y qué fichero corresponde a cada uno.
- input() = pide router, IP y usuario.
- getpass() = pide la contraseña sin mostrarla.
- urlopen() = descarga la configuración.
- ConnectHandler() = conecta por SSH.
- Cisco = filtra ciertas líneas y usa send_config_set().
- MikroTik = SFTP → import → borrar fichero temporal.
- VyOS = crear fichero → load → commit → save.
- FRR = crear fichero → vtysh -f → write memory.
- disconnect() = cierra la conexión.

20. Esquema rápido para recordar



21. Checklist antes del examen

- Comprobar que el entorno virtual tiene Netmiko y Paramiko instalados.
- Comprobar que la URL BASE apunta a la carpeta correcta.
- Comprobar que los nombres de los ficheros coinciden exactamente con GitHub.
- Comprobar que cada entrada de ROUTERS tiene el tipo Netmiko correcto.
- No escribir IP, usuario ni contraseña dentro del código si el ejercicio pide solicitarlos al usuario.
- Probar al menos una vez cada método de restauración que vaya a entrar en el examen.
- Mantener el bloque específico de cada sistema: Cisco, MikroTik, VyOS y FRR no restauran la configuración de la misma manera.

Nota: Este manual explica el script actual de la topología de monitorización que has mostrado. Los cambios indicados para otras topologías son una guía de adaptación: si el examen especifica otros formatos de configuración o un método concreto de restauración, debe seguirse lo que indique el enunciado.